#A If the origin is allowed then add the basic CORS headers to the response
#B If the origin is not allowed then reject the preflight request
#C For permitted preflight requests return a 204 No Content status
#D Preflight requests use the HTTP OPTIONS method and include the CORS request method header

To enable the CORS filter, you need to add it to the main method as a Spark `before()` filter, so that it runs before the request is processed. CORS preflight requests should be handled before your API requests authentication because credentials are never sent on a preflight request, so it would always fail otherwise. Open the Main.java file in your editor (it should be right next to the new CorsFilter.java file you just created) and find the main method. Add the following call to the main method right after the rate-limiting filter that you added in chapter 3:

```
var rateLimiter = RateLimiter.create(2.0d);    #A
before((request, response) -> {       #A
    if (!rateLimiter.tryAcquire()) {     #A
        halt(429);     #A
    }
});
before(new CorsFilter(Set.of("https://localhost:9999")));
#B
```

#A The existing rate-limiting filter.
#B The new CORS filter.

This ensures the new UI server running on port 9999 can make requests to the API. If you now restart the API server